

SQLite-Carver-Pro v1.9.2

Technical Reference Manual & Low-Level Forensic Engineering Specifications

 SEE & SQLCipher Decryption, WAL Slack Carving, Hex Inspection & Forensic Soundness

Comprehensive technical manual detailing physical page layout mechanics, transparent decryption of SQLCipher and SEE databases, encrypted WAL frame carving, raw unallocated and cell slack hex inspection, complete index overflow chain resolution, and forensic integrity protections.

Author: Dam-FOR3K

Version: 1.9.2 (Professional Forensic Edition)

Target Architecture: Standalone Air-Gapped Engine (Windows x86_64, Linux, macOS)

Methodological Standard: ISO/IEC 27037 Digital Evidence Handling & Forensic Soundness

1. Physical SQLite Architecture & Forensic Audit Areas

Unlike standard database management suites that depend on the official SQLite engine library (`sqlite3.dll`), **SQLite-Carver-Pro** operates at the binary level directly on raw disk pages. This low-level methodology bypasses database engine abstraction layers, allowing the recovery of unallocated, overwritten, fragmented, and anti-forensically concealed records.

1.1 The 100-Byte Database Header

The first 100 bytes of a database file (located exclusively on Page 1) define the global database parameters:

Offset	Size	Field Name	Forensic Significance
0	16 B	Magic Header String	Mandatory ASCII string: <code>SQLite format 3\0</code> . Validates authentic file signature.
16	2 B	Page Size	Physical page size (512 to 65,536 bytes, power of 2). Dictates all boundary math.
18	1 B / 1 B	Write / Read Version	File format version (1 = Conventional Rollback Journal, 2 = Write-Ahead Log / WAL mode).
20	1 B	Page Reserved Space	Critical Area: Reserved bytes at end of each page (0 to 255). Stores IV/HMAC in SQLCipher (48 bytes) or serves as an anti-forensic steganography vector.
24	4 B	File Change Counter	Incremented on each write transaction. Detects tampering and rollback anomalies.
28	4 B	Database Size in Pages	Number of valid pages. Immediately exposes <i>EOF Slack</i> (data appended beyond database bounds).
32	4 B	First Freelist Trunk Page	Pointer to the primary trunk page of the unallocated freelist chain.
36	4 B	Total Freelist Pages	Total count of decommissioned pages available for immediate reuse.
52	4 B	Largest Root Page / Vacuum	Auto-Vacuum mode: <code>NONE</code> , <code>FULL</code> , or <code>INCREMENTAL</code> .
56	4 B	Text Encoding	Default string serialization (1 = UTF-8, 2 = UTF-16le, 3 = UTF-16be).

1.2 B-Tree Page Types & Overflow Chains

Every physical database page is governed by one of four B-Tree page types identified by its first byte:

- **Table Leaf Page (`0x0D`):** Stores actual application records and integer RowID.
- **Table Interior Page (`0x05`):** Routing B-Tree containing navigation keys and child page pointers.
- **Index Leaf Page (`0x0A`):** Houses indexed column keys, row references, and `WITHOUT ROWID` tables.
- **Index Interior Page (`0x02`):** Index routing B-Tree pointing to child index pages.

v1.9.2 Enhancement: Accurate Index Overflow Chain Assembly

Pursuant to the official SQLite format specification, index cells employ a dedicated local payload limit: `max_local = ((U - 12) * 64) // 255 - 23` (where U represents usable page size). SQLite-Carver-Pro v1.9.2 correctly implements this formula and traverses all chained overflow pages on index B-Trees, guaranteeing full extraction of large keys without truncation.

1.3 Page Structure & Raw Unallocated Hex Inspection

Physical B-Tree pages are structured into distinct areas:

- **B-Tree Header:** Specifies page type, first freeblock offset, cell count, and content area start.
- **Cell Pointer Array:** Array of 16-bit integers referencing active cells.
- **Unallocated Space:** Contiguous byte gap between the end of the cell pointer array and the start of the cell content area.
- **Cell Slack:** Unallocated gaps between adjacent active cells within the cell content area.
- **Freeblocks:** Decommissioned cell fragments linked in an internal page freelist.

v1.9.2 Enhancement: Raw Residue Carving & Interactive Hex Inspection

Whenever unallocated space or inter-cell slack contains non-zero residual bytes (dirty remnants from partial overwrites or damaged records), the carving engine automatically yields a structured `[Raw Unallocated Fragment]` or `[Raw Slack Fragment]` record. Investigators can directly inspect these fragments inside the interactive Hex Viewer in both Desktop GUI and HTML forensic reports.

2. Unified Cryptographic Engine: SEE & SQLCipher

2.1 Cryptographic Architecture Comparison (SEE vs SQLCipher)

Modern digital forensics frequently encounters encrypted databases across mobile and desktop extractions:

Feature	SQLCipher (v3 / v4)	SQLite Encryption Extension (SEE)
Cipher & Mode	AES-256-CBC (per-page initialization vector)	AES-256-OFB or AES-128-OFB (Output Feedback)
Key Derivation	PBKDF2-HMAC-SHA512 (256,000 iter) or SHA1 (64,000 iter)	Direct SHA-256 / SHA-1 derivation or raw hex key
Reserve Bytes	48 bytes (16B IV + 20/32B HMAC)	0 or 16 bytes (nonce stored at end of page)
Page 1 Header	Bytes 0-15 overwritten by cryptographic salt	Bytes 16-23 stored in plaintext (geometry preserved)
WAL Encryption	Frame page payloads encrypted with AES-CBC	Frame page payloads encrypted with AES-OFB

2.2 Forensic Fingerprinting of SEE Databases

Because SQLite Encryption Extension (SEE) preserves bytes 16–23 in plaintext, **SQLite-Carver-Pro v1.9.2** detects SEE databases with **98% confidence** by verifying page sizes, format versions, and payload distribution ratios prior to passphrase input.

2.3 Transparent Decryption of Encrypted WAL Frames

During WAL operations, SQLite writes headers in plaintext while page data is encrypted. `WalDiffEngine` derives the master key and automatically decrypts each frame payload in memory using `decrypt_wal_frame_page()`, facilitating seamless extraction of superseded records and mutations.

3. Advanced Forensic Innovations & Methodological Soundness

3.1 WAL Slack Space Carving (Superseded Frames)

When SQLite checkpoints, the WAL file is logically reset by altering transaction salt (`salt1`). Frames situated past the active commit boundary retaining older salts constitute **WAL Slack Space**. SQLite-Carver-Pro automatically harvests these historical pre-checkpoint records marked with `wal_slack` provenance.

3.2 Multi-Version Page Reconstitution

Across multiple WAL cycles, pages are often modified repeatedly. The engine constructs a full chronological tree of row mutations (initial values, intermediate modifications, and final deletions).

3.3 Anti-Forensics: PRAGMA secure_delete Detection

When `PRAGMA secure_delete = ON` is active, deleted cells are zeroed out (`0x00`). The integrity module calculates zero-byte density across freeblocks; if over **90% of bytes are zeroed**, it raises `PRAGMA_SECURE_DELETE_DETECTED`.

3.4 Forensic Soundness: Anti-OOM Safeguards & Anti-Injection

- **Anti-OOM Protection (Large Databases):** If memory mapping (`mmap`) fails on files larger than 256 MB, the tool terminates cleanly with an explanatory error rather than exhausting host system RAM.
- **CSV Formula Injection Sanitization:** All exported tabular fields beginning with dynamic spreadsheet formula operators (`=`, `+`, `-`, `@`) are sanitized with a leading apostrophe to prevent DDE code execution in Microsoft Excel.
- **GUI Thread-Safe Event Dispatching:** Background carving threads strictly marshal progress events through the main GUI event loop, eliminating segmentation faults.

4. CLI Command Reference & Desktop GUI

4.1 Modern Desktop GUI Application

```
# Launch modern interactive forensic desktop GUI
sqlite-carver gui

# Open encrypted database directly
sqlite-carver gui evidence.db --key "PassphraseOrHexKey"
```

The unified **Decryption Key (SQLCipher / SEE)** input field accepts text passphrases or raw 32/64-character hexadecimal keys.

4.2 CLI Command Reference

```
# 1. Complete forensic carve with transparent decryption and HTML report
sqlite-carver carve evidence.db --key "secret_passphrase" --export report.html

# 2. Extract records from WAL Slack space (superseded frames)
sqlite-carver wal-diff evidence.db evidence.db-wal --slack --key "secret_passphrase"

# 3. Raw Page Hunter on unallocated memory dump without SQLite header
sqlite-carver carve-raw memory.dump --workers 8 --export carved.sqlite

# 4. Deep search across records, embedded BLOBs, and WAL
sqlite-carver search evidence.db "target_keyword" --key "secret_passphrase"
```

Certified Standalone Air-Gapped Distribution (v1.9.2)

Packaged as a single self-contained 64-bit binary (`sqlite-carver-v1.9.2.exe`) engineered for deployment on isolated forensic workstations without Python runtime dependencies.